

Componente A - Embeddings y Similitud: la Geometria del Lenguaje

Fase 3 (Modulo 3, Persistencia de datos y vector DBs). Se seleccionan 5 oraciones que expresan el mismo concepto tecnico (despliegue independiente de microservicios) con vocabulario distinto entre si, mas 2 oraciones trampa que comparten palabras clave literales pero significan algo totalmente distinto e irrelevante. Los embeddings se calculan con el mismo modelo local usado en los Componentes C y D (Sentence Transformers all-MiniLM-L6-v2 via ONNX, sin API key), para que el analisis sea consistente con el resto del sistema RAG.

1. Oraciones analizadas

Concepto: despliegue independiente de microservicios (5 oraciones, vocabulario distinto):

- [concepto_1] Publicar cada componente del sistema por separado en produccion reduce el riesgo de una actualizacion global fallida.
- [concepto_2] La orquestacion de contenedores permite que distintos servicios se actualicen de forma independiente sin afectar al resto de la aplicacion.
- [concepto_3] Empaquetar la logica de negocio en unidades autonomas facilita liberar nuevas versiones sin detener el resto de la plataforma.
- [concepto_4] Cada modulo del backend se libera segun su propio ciclo, minimizando el impacto de una falla puntual en produccion.
- [concepto_5] El equipo de infraestructura promueve builds independientes por servicio para acelerar la entrega continua.

Oraciones trampa (comparten palabras clave, significado no relacionado):

- [trampa_1] El servicio de micro-limpieza del edificio es excelente y siempre llega puntual.
- [trampa_2] Los contenedores de reciclaje del barrio se despliegan cada lunes en la vereda.

2. Calculo de Similitud Coseno (scikit-learn)

Fragmento real del script embeddings_similitud.py (se ejecuto de verdad; los resultados de la seccion 3 son la salida real de esta corrida, no valores de ejemplo):

```
from chromadb.utils import embedding_functions
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def calcular_embeddings(oraciones):
    embedding_fn = embedding_functions.DefaultEmbeddingFunction()
    vectores = embedding_fn(oraciones)
    return np.array(vectores)

vectores = calcular_embeddings(textos) # 7 oraciones (5 concepto + 2 trampa)
matriz = cosine_similarity(vectores) # matriz de similitud 7x7
```

3. Resultados (corrida real)

Matriz de similitud coseno entre las 7 oraciones:

	concepto_1	concepto_2	concepto_3	concepto_4	concepto_5	trampa_1	trampa_2
concepto_1	1.000	0.392	0.468	0.642	0.446	0.356	0.422
concepto_2	0.392	1.000	0.529	0.436	0.600	0.444	0.424
concepto_3	0.468	0.529	1.000	0.464	0.451	0.398	0.435
concepto_4	0.642	0.436	0.464	1.000	0.513	0.520	0.584
concepto_5	0.446	0.600	0.451	0.513	1.000	0.513	0.461
trampa_1	0.356	0.444	0.398	0.520	0.513	1.000	0.469
trampa_2	0.422	0.424	0.435	0.584	0.461	0.469	1.000

Similitud promedio ENTRE las 5 oraciones de concepto: 0.494

Similitud promedio de las TRAMPAS hacia el concepto: 0.456

El promedio dentro del cluster de concepto es mayor que el de las trampas hacia ese cluster, pese a que las oraciones de concepto no repiten vocabulario entre si y las trampas si comparten palabras literales con ellas.

Evidencia mas fina (la comparacion relevante para el objetivo del ejercicio): se enfrenta cada trampa contra 'concepto_2' (con quien comparte literalmente la palabra 'servicio(s)'/contenedores') y se compara esa similitud puntual contra el promedio de 'concepto_2' con sus propios pares semanticos reales (concepto_1/3/4/5, que no comparten esas palabras exactas con 'concepto_2'):

[CORRECTO] trampa_1 vs concepto_2 (palabra compartida) = 0.444 < promedio de concepto_2 con sus pares reales = 0.489

[CORRECTO] trampa_2 vs concepto_2 (palabra compartida) = 0.424 < promedio de concepto_2 con sus pares reales = 0.489

Conclusion: en ambos casos la oracion trampa es MENOS similar a concepto_2 de lo que concepto_2 es a sus verdaderos pares semanticos, pese a compartir una palabra clave literal con el. El embedding esta capturando significado, no coincidencia lexica superficial.

4. Diagrama de flujo: busqueda semantica

Desde que el usuario ingresa una consulta hasta que el sistema devuelve el documento mas parecido:

